Dispositivos Conectados
Conectividade Bluetooth Low Energy

2026/2027

Authors: Paulo C. Bartolomeu

Versão 1.0
27 de outubro de 2026

Histórico de versões

Versão	Data	Alterações
1.0	2026-10-27	Versão inicial.

1 Objetivos

- Compreender a arquitetura do Bluetooth Low Energy (BLE), os papéis GAP e os processos de anúncio, descoberta e ligação;
- Compreender o modelo de dados GATT e as operações de leitura, escrita, notificação e indicação;
- Utilizar a pilha NimBLE do ESP-IDF, baseada em funções de tratamento de eventos;
- Implementar no ESP32-C6 um servidor GATT com o *Environmental Sensing Service* e um serviço proprietário de configuração protegido por emparelhamento.

2 Introdução

2.1 O Bluetooth LE no contexto dos *Dispositivos Conectados*

Na aula anterior, o ESP32-C6 foi ligado a uma rede IP através do Wi-Fi, o que pressupõe um *Access Point* e tem um custo energético elevado. Em muitas situações, porém, um dispositivo precisa de comunicar com um utilizador que está **ao pé dele**, sem infraestrutura: consultar as leituras de um sensor numa estufa sem cobertura Wi-Fi, configurar as credenciais de rede de um equipamento acabado de instalar ou diagnosticar uma avaria. O interlocutor natural é o telemóvel, e a tecnologia que todos disponibilizam para este fim é o **Bluetooth Low Energy**.

O BLE foi introduzido na versão 4.0 da especificação Bluetooth (2010), mantida pelo *Bluetooth Special Interest Group* (SIG), e não é compatível com o Bluetooth "clássico" (BR/EDR). Foi desenhado para dispositivos que trocam pequenas quantidades de dados de forma esporádica e passam a maior parte do tempo com o rádio desligado. Ao contrário do Wi-Fi, **não transporta IP de forma nativa**: os dados são organizados num modelo próprio de atributos (GATT), e é o telemóvel (ou um *gateway*) que, se necessário, os encaminha para a Internet.

O BLE usa a banda de 2,4 GHz, dividida em **40 canais de 2 MHz**. Os canais 37, 38 e 39 (2402, 2426 e 2480 MHz) são reservados para anúncios e situam-se nas margens e nos intervalos dos canais Wi-Fi 1, 6 e 11; os restantes 37 transportam os dados das ligações, com saltos de frequência adaptativos (*Adaptive Frequency Hopping*).

2.2 Arquitetura protocolar

A pilha BLE divide-se num *controller* e num *host*, que comunicam através da *Host Controller Interface* (HCI):

Tabela 2: Camadas da pilha Bluetooth LE.

Parte	Camada	Função
Controller	PHY	Modulação GFSK a 1 Mbit/s, 2 Mbit/s ou *Coded* (125/500 kbit/s)
	Link Layer	Anúncios, ligações, temporização, saltos de frequência, cifra AES-CCM
Host	L2CAP	Multiplexagem dos protocolos superiores
	ATT / GATT	Acesso a atributos e sua organização em serviços e características
	SMP	Emparelhamento e distribuição de chaves
	GAP	Papéis, anúncios, descoberta, ligação e modos de segurança

No ESP32-C6, o *controller* é uma biblioteca binária da Espressif e o *host* é escolhido no `menuconfig` (`Component config > Bluetooth > Host`): **NimBLE**, uma pilha leve, originária do projeto Apache Mynewt e dedicada ao BLE, ou **Bluedroid**, que ocupa mais memória. O valor por omissão do ESP-IDF é o Bluedroid; os exemplos desta aula selecionam o NimBLE no ficheiro `sdkconfig.defaults`.

2.3 GAP: papéis, anúncios e endereços

O GAP define quatro papéis: ***broadcaster*** (emite anúncios, sem aceitar ligações, como um *beacon*), ***observer*** (escuta anúncios), ***peripheral*** (anuncia-se e aceita ligações; no nosso caso, o ESP32-C6) e ***central*** (procura dispositivos e inicia ligações; tipicamente, o telemóvel).

Um periférico emite **pacotes de anúncio** nos três canais de anúncio, com um **intervalo de anúncio** entre 20 ms e 10,24 s, a que o *controller* soma um atraso aleatório de 0 a 10 ms. O tipo de pacote (PDU) indica o que o dispositivo aceita:

Tabela 3: Tipos de pacotes de anúncio (*legacy advertising*).

PDU	Aceita ligação	Aceita *scan request*	Utilização típica
`ADV_IND`	sim	sim	Periférico à espera de ligação
`ADV_SCAN_IND`	não	sim	*Beacon* com informação adicional
`ADV_NONCONN_IND`	não	não	*Beacon* puro
`ADV_DIRECT_IND`	só de um central	não	Religação rápida a um central conhecido

Os dados de anúncio estão limitados a **31 bytes**. Um central em pesquisa ativa pode enviar um *scan request*, a que o periférico responde com mais 31 bytes (a *scan response*). Ambos são formados por estruturas AD, cada uma com um byte de comprimento, um byte de tipo e os dados:

Tabela 4: Estruturas AD utilizadas nos exemplos.

Tipo	Nome	Tipo	Nome
`0x01`	*Flags*	`0x19`	*Appearance*
`0x03`	*Complete List of 16-bit Service UUIDs*	`0x1A`	*Advertising Interval* (unidades de 0,625 ms)
`0x09`	*Complete Local Name*	`0x1B`	*LE Bluetooth Device Address*
`0x0A`	*TX Power Level* (dBm)	`0x1C`	*LE Role*
`0x16`	*Service Data - 16-bit UUID* (UUID seguido de dados)	`0x24`	*URI*

Cada dispositivo é identificado por um endereço de 48 bits, que pode ser **público** (registado no IEEE, fixo), **aleatório estático** (gerado pelo dispositivo e mantido enquanto este está alimentado) ou **privado**, mudando periodicamente (tipicamente, a cada 15 minutos). Os telemóveis usam endereços privados **resolúveis**, que só podem ser associados à sua identidade por quem conheça a *Identity Resolving Key* (IRK), trocada no emparelhamento. Como os restantes campos numéricos, os endereços são transmitidos em ***little-endian*** (byte menos significativo primeiro).

2.4 Ligação e parâmetros de ligação

Para se ligar, o central responde a um anúncio com um pacote `CONNECT_IND`. A partir daí, os dois dispositivos encontram-se periodicamente nos canais de dados, em **eventos de ligação**, segundo três parâmetros definidos pelo central (o periférico pode pedir alterações, que o central aceita ou não):

Tabela 5: Parâmetros de uma ligação BLE.

Parâmetro	Unidade	Gama	Significado
Connection interval	1,25 ms	7,5 ms a 4 s	Período entre eventos de ligação
Peripheral latency	eventos	0 a 499	Eventos que o periférico pode ignorar se não tiver dados
Supervision timeout	10 ms	100 ms a 32 s	Tempo sem comunicação ao fim do qual a ligação é dada como perdida

O intervalo de ligação determina a latência, o débito e o consumo: com 1 s, o periférico acorda cerca de 130 vezes menos do que com 7,5 ms. O *supervision timeout* tem de ser superior a $(1 + \text{latência}) \times \text{intervalo} \times 2$. Cada ligação é identificada por um ***connection handle***, atribuído pelo *controller* local, com um papel semelhante ao AID no Wi-Fi.

2.5 GATT: serviços, características e descritores

Depois de ligados, o central (cliente GATT) acede à **tabela de atributos** do periférico (servidor GATT) através do ATT. Cada atributo tem um ***handle*** (número de 16 bits), um **tipo** (UUID), **permissões** e um **valor**. Os atributos organizam-se em **serviços**, que agrupam **características**; cada característica tem um atributo de declaração, um atributo de valor e, opcionalmente, **descritores**, dos quais o mais importante é o ***Client Characteristic Configuration Descriptor*** (CCCD), onde o cliente ativa notificações ou indicações.

Os UUID têm 16 bits quando atribuídos pelo Bluetooth SIG (forma abreviada de `0000xxxx-0000-1000-8000-00805F9B34FB`) e 128 bits nos serviços proprietários:

Tabela 6: UUID normalizados utilizados nos exemplos.

UUID	Tipo	Nome	UUID	Tipo	Nome
`0x1800`	Serviço	*Generic Access*	`0x181A`	Serviço	*Environmental Sensing*
`0x1801`	Serviço	*Generic Attribute*	`0x2A6E`	Característica	*Temperature*
`0x180D`	Serviço	*Heart Rate*	`0x2A6F`	Característica	*Humidity*
`0x2A37`	Característica	*Heart Rate Measurement*	`0x2902`	Descritor	CCCD (bit 0: notificar; bit 1: indicar)
`0x1815`	Serviço	*Automation IO*			

O cliente pode **ler** (*Read*) e **escrever** (*Write*, com resposta, ou *Write Without Response*). O servidor pode enviar **notificações**, sem confirmação, ou **indicações**, confirmadas pelo cliente, mas apenas a clientes que as tenham ativado no CCCD. A dimensão de cada pacote ATT é limitada pelo **MTU**, de 23 bytes por omissão (20 bytes úteis numa notificação), que o cliente pode negociar no início da ligação.

2.6 Modelo de programação NimBLE no ESP32-C6

O ESP32-C6 suporta Bluetooth LE 5 (sem Bluetooth clássico). O endereço público BLE é derivado do endereço MAC base do *chip*: por omissão, a interface Wi-Fi STA usa o endereço base, a SoftAP o base + 1 e o Bluetooth o base + 2.

Tal como no Wi-Fi, a pilha NimBLE é **assíncrona** e orientada a eventos, mas não usa o *event loop* do ESP-IDF: o *host* corre numa tarefa FreeRTOS própria e invoca as funções (*callbacks*) indicadas pela aplicação. Os exemplos seguem esta sequência (Figura 1):

1. `nvs_flash_init()`: inicializa a NVS, usada para a calibração do rádio e, opcionalmente, para guardar as chaves de emparelhamento;
2. `nimble_port_init()`: inicializa o *controller* e o *host*;
3. `gap_init()` e, nos exemplos com servidor GATT, `gatt_svc_init()` (funções dos exemplos): definem o nome do dispositivo (`ble_svc_gap_device_name_set()`) e registam a tabela de serviços (`ble_gatts_count_cfg()` e `ble_gatts_add_svcs()`);
4. `nimble_host_config_init()`: preenche a estrutura global `ble_hs_cfg` com as *callbacks* de sincronização (`sync_cb`) e de registo de atributos (`gatts_register_cb`) e, no exemplo `NimBLE_Security`, os parâmetros de segurança;
5. `xTaskCreate(nimble_host_task, ...)`: cria a tarefa do *host*, que executa `nimble_port_run()`;
6. quando o *host* e o *controller* sincronizam, `sync_cb` chama `adv_init()`, que determina o endereço e inicia os anúncios: `ble_gap_adv_set_fields()` (dados de anúncio), `ble_gap_adv_rsp_set_fields()` (*scan response*) e `ble_gap_adv_start()`, cujo penúltimo argumento é a função de tratamento de eventos GAP (`gap_event_handler()`).

Tabela 7: Eventos GAP do NimBLE utilizados nos exemplos.

Evento	Quando ocorre
`BLE_GAP_EVENT_CONNECT` / `_DISCONNECT`	Ligação estabelecida (ou tentativa falhada, campo `status`) / terminada (campo `reason`)
`BLE_GAP_EVENT_CONN_UPDATE`	Terminou uma atualização dos parâmetros da ligação (campo `status`)
`BLE_GAP_EVENT_MTU`	O MTU foi negociado
`BLE_GAP_EVENT_SUBSCRIBE`	O cliente ativou ou desativou notificações ou indicações
`BLE_GAP_EVENT_ENC_CHANGE`	A cifra da ligação foi ativada (ou falhou)
`BLE_GAP_EVENT_PASSKEY_ACTION`	O emparelhamento requer uma ação (por exemplo, mostrar um código)
`BLE_GAP_EVENT_REPEAT_PAIRING`	Um dispositivo já emparelhado pediu para voltar a emparelhar

No servidor GATT, cada característica tem uma **função de acesso** (`access_cb`), invocada quando um cliente a lê (`BLE_GATT_ACCESS_OP_READ_CHR`) ou escreve (`BLE_GATT_ACCESS_OP_WRITE_CHR`), e quando a pilha precisa do valor para uma notificação. Na leitura, acrescenta o valor a `ctxt->om` com `os_mbuf_append()`; na escrita, lê-o desse *buffer*. Devolve `0` ou um código de erro ATT (`BLE_ATT_ERR_*`), que é enviado ao cliente.

> **IMPORTANTE:** as funções de tratamento de eventos e de acesso correm, normalmente, na **tarefa do *host***: enquanto uma delas estiver bloqueada (por exemplo, numa leitura I²C), nenhum outro evento BLE é tratado. Devem limitar-se a ler ou atualizar valores em

memória. Exceção: quando a aplicação chama `ble_gatts_notify()` ou `ble_gatts_indicate()`, a função de acesso corre **na tarefa que fez a chamada**; se essa tarefa segurar um *mutex* que a função de acesso também tenta obter, o programa bloqueia. `ble_gatts_chr_updated()` apenas agenda o envio, feito depois pela tarefa do *host*.

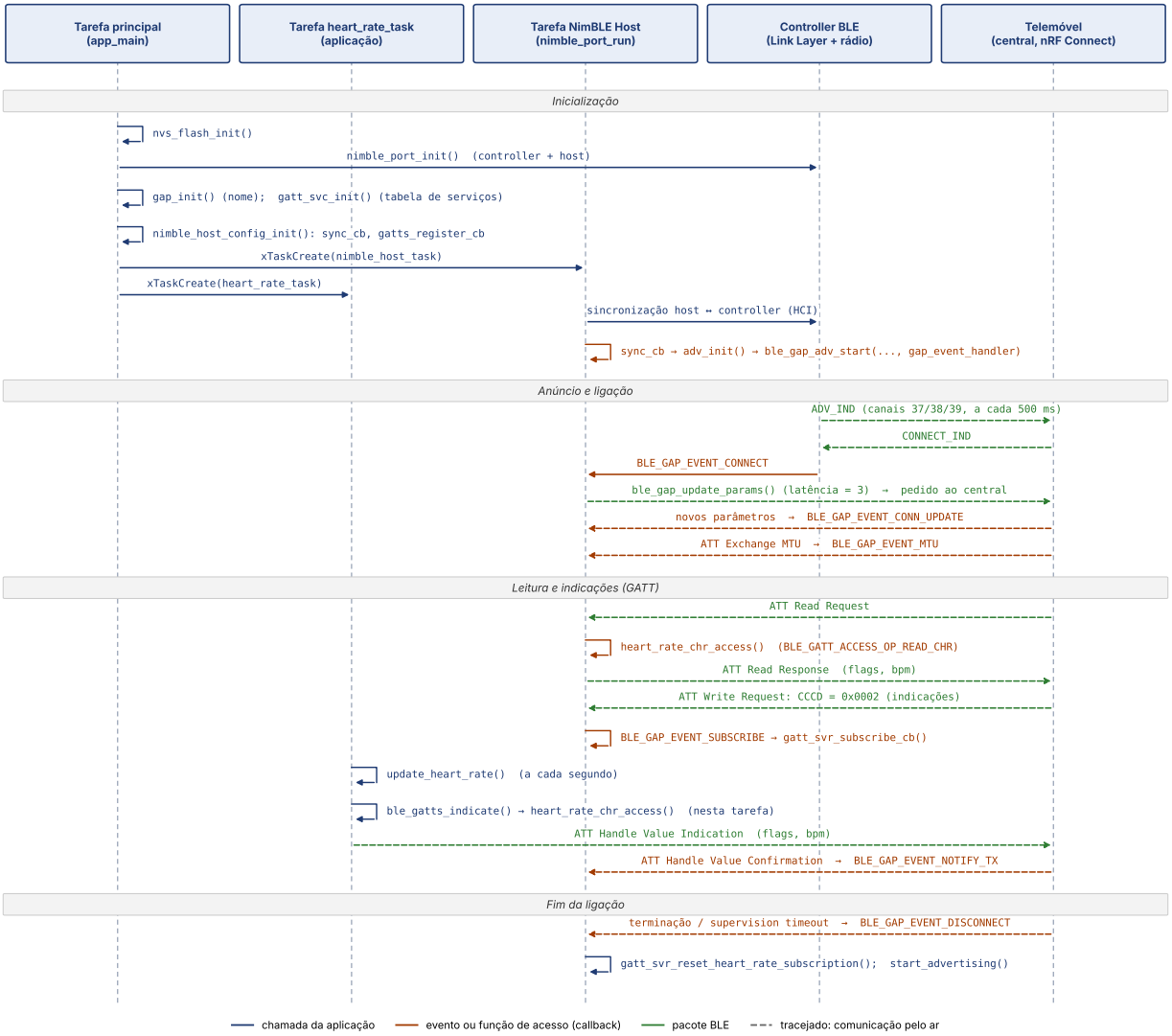


Figura 1: Sequência do exemplo `NimBLE_GATT_Server`

2.7 Segurança, energia e coexistência

Uma ligação BLE é, por omissão, transmitida **em claro**. Para a cifrar, os dispositivos **emparelham-se** (*pairing*), acordando uma chave através do SMP; se guardarem as chaves para ligações futuras, estabelecem uma ***bond***. O emparelhamento *LE Legacy* é vulnerável a escuta passiva; o ***LE Secure Connections*** (BLE 4.2 ou superior) usa troca de chaves ECDH e é usado pelo NimBLE sempre que o telemóvel o suporte (`ble_hs_cfg.sm_sc` é inicializado a 1 a partir de `CONFIG_BT_NIMBLE_SM_SC`). O **modelo de associação** depende das capacidades de entrada e saída dos dispositivos: no *Just Works* não há interação nem proteção contra *man-in-the-middle* (MITM); no *Passkey Entry* (um dispositivo

mostra um código de 6 dígitos que o utilizador introduz no outro) e no *Numeric Comparison*, a ligação fica **autenticada**.

No NimBLE, os requisitos de cada característica são definidos nas suas *flags*: `BLE_GATT_CHR_F_READ_ENC`/`_WRITE_ENC` exigem uma ligação cifrada e `BLE_GATT_CHR_F_READ_AUTHEN`/`_WRITE_AUTHEN` cifrada e autenticada. Estas *flags* **acrescentam requisitos, mas não permitem a operação**: sem `BLE_GATT_CHR_F_READ` (ou `_WRITE`), a característica não pode ser lida (ou escrita). Por omissão, as *bonds* **não são guardadas na NVS** e perdem-se quando a placa reinicia (opção `CONFIG_BT_NIMBLE_NVS_PERSIST`).

O consumo de um periférico BLE depende sobretudo do intervalo de anúncio, dos parâmetros de ligação e da potência de emissão (`Component config > Bluetooth > Controller Options > BLE default Tx power level`, +9 dBm por omissão). Para adormecer entre eventos de rádio, é ainda necessário ativar `CONFIG_BT_LE_SLEEP_ENABLE`, `CONFIG_PM_ENABLE` e `CONFIG_FREERTOS_USE_TICKLESS_IDLE` e chamar `esp_pm_configure()` com `light_sleep_enable = true` (exemplo `bluetooth/nimble/power_save`). As opções desse exemplo para o ESP32-C6 assumem um cristal de 32,768 kHz, que o DevKitC-1 não inclui: na placa da aula, mantenha `BLE low power clock source = Use main XTAL as RTC clock source.`

O ESP32-C6 tem um **único rádio de 2,4 GHz**, partilhado no tempo entre Wi-Fi, BLE e IEEE 802.15.4 (`CONFIG_ESP_COEX_SW_COEXIST_ENABLE`, ativa por omissão). Com Wi-Fi e BLE em simultâneo, os eventos de cada um podem ser adiados pelo outro, pelo que os parâmetros de ambas as ligações devem ter folga (exemplo `bluetooth/nimble/bleprph_wifi_coex`).

3 Trabalho Prático

O trabalho prático baseia-se no Guia de introdução ao Bluetooth LE do ESP-IDF, que acompanha os exemplos `ble_get_started`, na API de Referência NimBLE, na documentação do NimBLE e nos documentos do Bluetooth SIG *Assigned Numbers* e *GATT Specification Supplement*. Todos os exercícios devem ser realizados com o ESP-IDF **v6.1.0**.

Preparação:

- Instale o **nRF Connect for Mobile** (usado nas descrições) ou o **LightBlue**. No Android, conceda à aplicação a permissão "Dispositivos próximos" (e, em versões antigas, de localização).
- **Antes de compilar cada exemplo**, altere a macro `DEVICE_NAME` em `main/include/common.h` para um nome do grupo com **no máximo 10 caracteres** (por exemplo, `"DC-P2-G07"`); caso contrário, todas as placas da sala anunciam o mesmo nome. A razão do limite é analisada nas Questões 2, 3 e 9.
- Os exemplos usam `MINIMAL_BUILD` e declaram apenas os componentes de que precisam em `main/CMakeLists.txt`. Para usar o I²C, acrescente `esp_driver_i2c` a `PRIV_REQUIRES`. Os ficheiros `.c` novos em `main/src/` só são compilados depois de executar `idf.py reconfigure` (ou *Full Clean*).
- **O iOS e o macOS não dão às aplicações o endereço BLE nem os dados de anúncio em bruto.** As Questões 1 e 2 precisam dessa informação e devem ser realizadas com o nRF Connect em **Android** ou no PC: no **Windows 10 (2004+) e 11**, com o *script* `resources/ble_scan.py` (`pip install bleak`, depois `python ble_scan.py DC-P2-G07`), que mostra o endereço, o tipo de PDU, as estruturas AD em bruto e, a cada 5 s, o RSSI médio e o menor intervalo entre anúncios; no **Linux**, com `sudo btmon` durante uma pesquisa iniciada no `bluetoothctl` (`menu scan`, `transport le`,

`duplicate-data on`, `back`, `scan on`); no **macOS**, com o PacketLogger (*Additional Tools for Xcode*).

3.1 Descoberta: o ESP32-C6 como *Beacon*

O exemplo `NimBLE_Beacon` configura o ESP32-C6 como *broadcaster*, com anúncios não conectáveis (`BLE_GAP_CONN_MODE_NON`). Como o modo de descoberta é geral, o NimBLE usa o PDU `ADV_SCAN_IND`, o que permite pedir a *scan response*. Os dados de anúncio contêm as *flags*, o nome, a potência de emissão, a aparência e o papel LE; a *scan response* contém o endereço e o URI `https://espressif.com`. Como o intervalo não é definido, o NimBLE usa o valor por omissão, de 100 a 150 ms.

QUESTÃO 1: No VSCode crie um novo projeto a partir do exemplo `bluetooth -> ble_get_started -> nimble -> NimBLE_Beacon`, altere `DEVICE_NAME`, compile o programa e descarregue o *firmware* para o SoC. Deverá observar um *log* semelhante ao seguinte:

```
I (436) NimBLE_Beacon: nimble host task has been started!
I (436) main_task: Returned from app_main()
I (446) NimBLE_Beacon: device address: BE:C0:3B:81:8C:58
I (446) NimBLE: GAP procedure initiated: advertise;
I (456) NimBLE: disc_mode=2
I (456) NimBLE:  adv_channel_map=0 own_addr_type=0 adv_filter_policy=0 adv_itvl_min=0 adv_itvl_max=0
I (466) NimBLE_Beacon: advertising started!
```

Localize o dispositivo no nRF Connect (separador **SCANNER**) ou na ferramenta do PC.

a) Compare o endereço apresentado na consola com o apresentado pela aplicação. Porque é que os bytes surgem por ordem inversa (consulte `format_addr()` em `main/src/gap.c`)? Que tipo de endereço é usado (`own_addr_type=0`)?

b) Que relação existe entre este endereço e o endereço MAC da interface STA da mesma placa, obtido na aula de Wi-Fi?

c) Porque é que a aplicação não apresenta o botão **CONNECT**, mas consegue mostrar o URI e o endereço, que estão na *scan response* (Tabela 3)?

QUESTÃO 2: Observe os dados de anúncio em bruto (no nRF Connect para Android, botão **RAW**).

a) Decomponha os dados de anúncio e a *scan response* nas estruturas AD (Tabela 4). Explique a representação da aparência (`BLE_GAP_APPEARANCE_GENERIC_TAG` vale `0x0200`) e do endereço.

b) Calcule os bytes ocupados nos dois blocos. Porque é que o endereço e o URI estão na *scan response*? Qual é o comprimento máximo do nome neste exemplo, e porque é que a preparação recomenda um nome mais curto? Confirme-o com um nome de 20 caracteres (`failed to set advertising data, error code: 4`, isto é, `BLE_HS_EMSGSIZE`).

c) Defina um intervalo de anúncio de 1 s (`adv_params.itvl_min` e `itvl_max` iguais a `BLE_GAP_ADV_ITVL_MS(1000)`) e compare o intervalo estimado pela aplicação e o tempo de descoberta antes e depois. Repita as medições de RSSI a 0,5 m, 2 m e 5 m com +9 dBm e com 0 dBm (média de várias leituras). Que compromissos existem entre intervalo, potência, consumo, alcance e latência de descoberta?

Um *beacon* pode também transmitir dados a todos os *observers* próximos, sem ligação, através da estrutura *Service Data*. No NimBLE, esta é preenchida com os campos `svc_data_uuid16` e `svc_data_uuid16_len` de `struct ble_hs_adv_fields`, num vetor com o UUID de 16 bits (em *little-endian*) seguido dos dados.

O TC74 partilha o barramento I²C do DHT20 (`SDA` no `GPIO 6`, `SCLK` no `GPIO 7`, com *pull-ups* de 4,7 kΩ; no encapsulamento TO-220, os pinos 2, 3, 4 e 5 são `SDA`, `GND`, `SCLK` e `VDD`). Responde num endereço entre `0x48` e `0x4F`, conforme a variante (`TC74A5`: `0x4D`), e fornece a temperatura no registo `0x00`, como inteiro de 8 bits com sinal, em °C. Suporta, no máximo, **100 kHz**: registe-o com `scl_speed_hz = 100000`.

> **QUESTÃO 3:** Altere o exemplo `NimBLE_Beacon` de modo a que os anúncios transportem a temperatura do TC74, atualizada a cada 2 segundos. Requisitos:
>
> - o TC74 é lido com o *driver* `driver/i2c_master.h` numa tarefa própria, que imprime cada valor e, se a leitura falhar, imprime um aviso e mantém o valor anunciado;
> - a temperatura é anunciada numa estrutura *Service Data* com o UUID `0x181A`, num byte com sinal;
> - após cada leitura, a tarefa chama `ble_gap_adv_set_fields()` com **todos** os campos (a função substitui o conteúdo completo, sem interromper os anúncios), apenas depois de o *host* ter sincronizado (`ble_hs_synced()`), e sem conflito de acesso com `start_advertising()`.
>
> A estrutura *Service Data* ocupa 5 bytes: verifique se cabe com o seu nome e, se necessário, retire um campo. Confirme no nRF Connect que o valor acompanha a temperatura. Que vantagens e limitações tem esta forma de disponibilizar uma medição, face a uma ligação?

3.2 Ligação: o ESP32-C6 como periférico

O exemplo `NimBLE_Connection` usa anúncios conectáveis (`ADV_IND`), a cada 500 ms, e acrescenta o intervalo de anúncio à *scan response*, que fica completa (31 bytes). Em `BLE_GAP_EVENT_CONNECT`, imprime os dados da ligação, acende o LED (GPIO 8, componente `espressif/led_strip`) e pede ao central uma latência de 3 eventos com `ble_gap_update_params()`; em `BLE_GAP_EVENT_DISCONNECT`, apaga o LED e **volta a anunciar**.

> **QUESTÃO 4:** Crie um projeto a partir do exemplo `NimBLE_Connection`, altere `DEVICE_NAME` e ligue-se a partir do nRF Connect (**CONNECT**). Deverá observar o LED a acender e um *log* semelhante ao seguinte:

```
I (36367) NimBLE_Connection: connection established; status=0
I (36367) NimBLE_Connection: connection handle: 0
I (36367) NimBLE_Connection: device id address: type=0, value=BE:C0:3B:81:8C:58
I (36377) NimBLE_Connection: peer id address: type=1, value=7F:BE:AD:66:6F:45
I (36377) NimBLE_Connection: conn_itvl=36, conn_latency=0, supervision_timeout=500, encrypted=0, ...
...
I (37007) NimBLE_Connection: connection updated; status=0
...
I (37017) NimBLE_Connection: conn_itvl=36, conn_latency=3, supervision_timeout=500, encrypted=0, ...
```

a) Identifique o *connection handle* e os parâmetros da ligação antes e depois da atualização, convertendo-os para milissegundos (Tabela 5).

b) Que tipo de endereço usa o telemóvel (analise o campo `type` e os dois bits mais significativos do endereço)? O endereço mantém-se se se desligar e voltar a ligar? Tenha em conta que o telemóvel só o muda periodicamente.

c) Com o telemóvel ligado, um segundo telemóvel encontra o dispositivo? Porquê? Que alteração permitiria até três ligações simultâneas (`CONFIG_BT_NIMBLE_MAX_CONNECTIONS` vale 3)?

QUESTÃO 5: Termine a ligação no nRF Connect (**DISCONNECT**) e observe `disconnected from peer; reason=...`. Volte a ligar-se e provoque uma **perda** da ligação, afastando o telemóvel ou envolvendo-o em papel de alumínio (ao desligar o Bluetooth, muitos telemóveis terminam a ligação de forma ordenada).

a) Os erros HCI são somados a `BLE_HS_ERR_HCI_BASE` (`0x200`). Descodifique as causas observadas (por exemplo, `531` e `520`) com base na *Core Specification* (Vol. 1, Parte F).

b) Quanto tempo decorre entre a perda de comunicação e o evento `BLE_GAP_EVENT_DISCONNECT`? Relacione-o com o *supervision timeout*.

c) Compare com a Questão 3 da aula de Wi-Fi (`bcn_timeout`, `Maximum retry`): quem toma a iniciativa de restabelecer a ligação, e o ESP32-C6 deixa de estar disponível em algum momento?

QUESTÃO 6: Altere o pedido feito em `BLE_GAP_EVENT_CONNECT` para um intervalo entre 500 e 1000 ms, latência 0 e *supervision timeout* de 6 s (macros `BLE_GAP_CONN_ITVL_MS()` e `BLE_GAP_SUPERVISION_TIMEOUT_MS()`). Que parâmetros foram efetivamente aplicados? O central pode recusar (`status=571`, *Unacceptable Connection Parameters*) ou aplicar outros valores, e alguns telemóveis fazem as suas próprias atualizações no início da ligação. Que impacto teriam estes parâmetros no consumo e no tempo de resposta a uma leitura?

3.3 Troca de dados: servidor GATT

O exemplo `NimBLE_GATT_Server` define, em `gatt_svr_svcs[]` (`main/src/gatt_svc.c`), dois serviços: o *Heart Rate* (`0x180D`), com a característica *Heart Rate Measurement* (`0x2A37`, leitura e indicação), cujo valor (1 byte de *flags* e o ritmo, entre 60 e 80 bpm) é atualizado a cada segundo por `heart_rate_task()` e indicado com `ble_gatts_indicate()`; e o *Automation IO* (`0x1815`), com uma característica proprietária de 128 bits, só de escrita, que acende ou apaga o LED.

QUESTÃO 7: Crie um projeto a partir do exemplo `NimBLE_GATT_Server`, altere `DEVICE_NAME` e, em `gatt_svr_register_cb()`, substitua `ESP_LOGD` por `ESP_LOGI`. Ligue-se no nRF Connect, leia a característica *Heart Rate Measurement* e ative as indicações. Deverá observar um *log* semelhante ao seguinte:

```
I (466) NimBLE_GATT_Server: registered service 0x1800 with handle=<h>
...
I (476) NimBLE_GATT_Server: registered service 0x180d with handle=<h>
I (476) NimBLE_GATT_Server: registering characteristic 0x2a37 with def_handle=<h> val_handle=<h>
I (476) NimBLE_GATT_Server: registered service 0x1815 with handle=<h>
I (486) NimBLE_GATT_Server: registering characteristic 00001525-1212-efde-1523-785feabcd123 with ...
...
I (12576) NimBLE_GATT_Server: mtu update event; conn_handle=0 cid=4 mtu=256
I (18406) NimBLE_GATT_Server: characteristic read; conn_handle=0 attr_handle=<h>
I (24116) NimBLE_GATT_Server: subscribe event; conn_handle=0 attr_handle=<h> reason=1 prevn=0 curn=0 previ=0
    curi=1
I (24896) NimBLE_GATT_Server: heart rate updated to 71
I (24896) NimBLE_GATT_Server: characteristic read by nimble stack; attr_handle=<h>
I (24906) NimBLE_GATT_Server: heart rate indication sent!
```

a) Construa a tabela de atributos dos dois serviços da aplicação (*handle*, UUID, significado). O CCCD é acrescentado pela pilha sem passar por `gatt_svr_register_cb()`: deduza o seu *handle*. Porque é que a característica do LED não tem CCCD? De onde surgem os serviços `0x1800` e `0x1801`?

b) Interprete os dois bytes lidos e o evento `subscribe` (`reason=1`, `curi=1`). Que valor foi escrito no CCCD?

c) Porque é que a função de acesso é invocada "*by nimble stack*" antes de cada indicação, e em que tarefa corre?

d) A especificação do *Heart Rate Service* prevê apenas notificações para esta característica. Que vantagem e que custo teriam indicações num sensor que envia uma medição por segundo?

QUESTÃO 8: No serviço *Automation IO*, localize a característica `00001525-1212-EFDE-1523-785FEABCD123`.

a) Escreva `01` e `00` e observe o LED. Escreva `0101`: que erro recebe o telemóvel e quem o gera? Porque é que a aplicação não oferece a leitura, e que erro devolveria a pilha a um pedido de leitura (`ble_att_svr_check_perms()`)? Use `host/ble_att.h`.

b) Compare o UUID com a declaração `BLE_UUID128_INIT(...)` de `led_chr_uuid`. Por que ordem são indicados os bytes?

c) No projeto, o serviço de configuração tem o UUID `E0C0DE00-0000-4000-8000-00000000XXXX`, em que `XXXX` é o `node_id` em hexadecimal, e as características alteram o primeiro grupo para `E0C0DE01`, `E0C0DE02`, etc. Escreva as declarações `BLE_UUID128_INIT(...)` do serviço e da primeira característica para o seu grupo (para `node_id` = 1022 = `0x03FE`, a do serviço começa por `0xFE, 0x03, 0x00, ...`).

3.3.1 O *Environmental Sensing Service*

As características *Temperature* (`0x2A6E`) e *Humidity* (`0x2A6F`) do ESS (`0x181A`) são inteiros de 16 bits, em *little-endian*, com resolução de 0,01, e reservam um valor para "desconhecido":

Tabela 8: Codificação das características do ESS (*GATT Specification Supplement*).

Medição	Valor	Inteiro	Bytes transmitidos	"Desconhecido"
Temperatura	23,45 °C	`sint16` 2345 = `0x0929`	`29 09`	`0x8000`
Humidade	48,20 %	`uint16` 4820 = `0x12D4`	`D4 12`	`0xFFFF`

Como o ESP32-C6 é *little-endian*, a função de acesso pode usar `os_mbuf_append(ctxt->om, &valor, sizeof(valor))` com uma variável `int16_t` ou `uint16_t`. Para notificar todos os clientes subscritos, basta chamar `ble_gatts_chr_updated(val_handle)` a partir de qualquer tarefa.

QUESTÃO 9: Altere o programa da Questão 7, substituindo o serviço *Heart Rate* pelo ESS, com as características *Temperature* e *Humidity* medidas pelo DHT20, com leitura e notificação. Requisitos:

- uma tarefa de aquisição, que substitui `heart_rate_task()`, mede a cada 2 segundos com `driver/i2c_master.h` e guarda a última amostra válida numa estrutura protegida por um *mutex*; as funções de acesso devolvem esses valores e **não acedem ao sensor** (justifique, comparando com a Questão 9 da aula de Wi-Fi);
- após cada medição válida, a tarefa **liberta o *mutex*** e notifica as duas características com `ble_gatts_chr_updated()`; remova `send_heart_rate_indication()` e `gatt_svr_subscribe_cb()`;
- sem amostra válida, a leitura devolve o valor "desconhecido" (Tabela 8); uma medição falhada imprime um aviso e mantém os últimos valores;
- cada leitura e cada notificação são registadas na consola com os valores enviados;
- o UUID `0x181A` é incluído nos dados de anúncio (campos `uuids16`, `num_uuids16` e `uuids16_is_complete`), ocupando 4 bytes: a *scan response* está completa, pelo que, se necessário, deve retirar um campo dos dados de anúncio.

Verifique os valores e o período das notificações no nRF Connect. Compare esta solução com o servidor HTTPS da aula de Wi-Fi, quanto a quem inicia a comunicação e à quantidade de dados por medição.

3.3.2 Serviço proprietário de configuração e segurança

Um dispositivo no terreno deve permitir alterar parâmetros, como o período de amostragem, sem ser reprogramado, através de um **serviço proprietário**. Como qualquer pessoa ao alcance do rádio se pode ligar, as características de configuração devem exigir uma ligação cifrada e autenticada, e as que recebem dados sensíveis não devem ser legíveis. O exemplo `NimBLE_Security` usa o modelo *Passkey Entry*: declara que o dispositivo só tem ecrã (`ble_hs_cfg.sm_io_cap = BLE_HS_IO_DISPLAY_ONLY`) e pede *bonding* e proteção MITM (`sm_bonding = 1`, `sm_mitm = 1`); em `BLE_GAP_EVENT_PASSKEY_ACTION`, gera um código de 6 dígitos, imprime-o na consola e entrega-o à pilha com `ble_sm_inject_io()`.

QUESTÃO 10: Acrescente ao programa da Questão 9 um serviço com o UUID da Questão 8 c), protegido por emparelhamento. Requisitos:

- `E0C0DE01-...`: **período de amostragem** em segundos (`uint16`, *little-endian*), com leitura e escrita. Rejeite escritas com dimensão diferente de 2 bytes (`BLE_ATT_ERR_INVALID_ATTR_VALUE_LEN`) e valores fora de [1, 3600] (`BLE_ATT_ERR_VALUE_NOT_ALLOWED`). Obtenha a dimensão com `OS_MBUF_PKTLEN(ctxt->om)` (e não com `om_len`, que só conta o primeiro *buffer*) e copie o valor com `ble_hs_mbuf_to_flat()`. A tarefa de aquisição usa o novo período a partir da medição seguinte, obtendo-o através de `get_sampling_period()`;
- `E0C0DE02-...`: **código de acesso** de texto, até 32 caracteres, **apenas de escrita**. O valor não termina em `'\0'` e nunca é impresso (regista-se apenas a sua dimensão);
- *flags*: `READ | READ_ENC | WRITE | WRITE_ENC | WRITE_AUTHEN` no período e `WRITE | WRITE_ENC | WRITE_AUTHEN` no código (prefixo `BLE_GATT_CHR_F_`); o ESS continua acessível sem emparelhamento;
- copie do `NimBLE_Security` a configuração do *Security Manager* e o tratamento de `BLE_GAP_EVENT_PASSKEY_ACTION`, `BLE_GAP_EVENT_ENC_CHANGE` (onde deve imprimir os dados da ligação) e `BLE_GAP_EVENT_REPEAT_PAIRING`, mas **não** a função `set_random_addr()`.

Antes de testar, apague emparelhamentos anteriores e, no nRF Connect para Android, use **Refresh device cache**.

a) Sem emparelhar, leia o ESS e tente escrever o período. O que acontece? Introduza o código mostrado na consola e verifique `encrypted`, `authenticated` e `bonded`. Que modelo

de associação foi usado, e porquê?

b) Altere o período para 10 s e verifique as notificações; tente escrever 0 e 5000.

c) Reinicie a placa (**RST**) e volte a ligar-se. A ligação volta a ser cifrada sem código? Ative `Persist the BLE Bonding keys in NVS` (`Component config > Bluetooth > NimBLE Options > Security (SMP) > Enable BLE SM feature`), apague a *bond* no telemóvel e repita. Porque é que, no exemplo `NimBLE_Security`, esta opção não bastaria (analise `set_random_addr()` e `ble_hs_id_gen_rnd()`)?

d) Um atacante passivo que capture o emparelhamento e a escrita consegue obter o código? E com *Just Works*, ou com *Passkey Entry* em *LE Legacy*?

e) No projeto, os valores das características proprietárias têm de incluir um campo de versão de formato. Proponha um formato para o período que cumpra este requisito.

O mecanismo da Questão 10 é a base do provisionamento em campo. A Espressif disponibiliza uma implementação completa no componente `espressif/network_provisioning` (obtido pelo gestor de componentes), usada pelas aplicações *ESP BLE Provisioning*.

4 Resolução de problemas

Sintoma	Causa provável	Ação
O dispositivo não aparece no telemóvel	Aplicação sem permissões; Bluetooth desligado	Conceder "Dispositivos próximos"; ativar o Bluetooth
`failed to set advertising data, error code: 4`	Dados de anúncio com mais de 31 bytes (`BLE_HS_EMSGSIZE`)	Encurtar o nome ou retirar campos (Questões 2, 3 e 9)
O iPhone ou o Mac não mostra o endereço nem os dados em bruto	Limitação do iOS e do macOS	Usar Android ou a ferramenta do PC (preparação)
`driver/i2c_master.h: No such file or directory`	`MINIMAL_BUILD` sem o componente I²C	Acrescentar `esp_driver_i2c` a `PRIV_REQUIRES`
`undefined reference` a funções de um ficheiro novo	Lista de ficheiros obtida por `GLOB_RECURSE` antes de o ficheiro existir	`idf.py reconfigure` ou *Full Clean*
O TC74 não responde	Frequência acima de 100 kHz ou endereço errado	`scl_speed_hz = 100000`; confirmar a variante `TC74Ax`
O LED não acende	`Blink LED type` ou GPIO errados	`Example Configuration`: `LED strip` e GPIO 8
Eventos BLE atrasados quando o sensor é lido	Leitura I²C numa função de acesso ou de eventos	Ler numa tarefa própria (Questão 9)
O programa bloqueia ao notificar	`ble_gatts_notify()`/`_indicate()` com o *mutex* obtido	Libertar o *mutex* antes; usar `ble_gatts_chr_updated()`
Read Not Permitted numa característica com `READ_ENC`	Falta `BLE_GATT_CHR_F_READ` (ou `_WRITE`)	Acrescentar a *flag* base (Questão 10)
O telemóvel mostra serviços antigos	Tabela de atributos em *cache*	**Refresh device cache** (Android); "Esquecer dispositivo" (iOS)

Sintoma	Causa provável	Ação
Emparelhamento falha ou é pedido de novo após reiniciar	*Bonds* não guardadas na NVS	Ativar `Persist the BLE Bonding keys in NVS`; apagar a *bond* no telemóvel

5 Acknowledgements

Originally authored by Paulo C. Bartolomeu